BAEKSEOK UNIVERSITY

Chap 11. Function

Contents

Functions

https://github.com/kkh3363/C_Lang

- Functions
  - A function is a block of code which only runs when it is called.
  - You can pass data, known as parameters, into a function.
  - Functions are used to perform certain actions, and they are important for reusing code: Define the code once, and use it many times.

Functions

https://github.com/kkh3363/C_Lang

Predefined Functions

- So it turns out you already know what a function is.
- For example, **main()** is a function, which is used to execute code, and **printf()** is a function; used to output/print text to the screen:

```c
int main() {
  printf("Hello World!");
  return 0;
}
```

Functions

https://github.com/kkh3363/C_Lang

- Create a Function
  - predefine function (once)
  - function define (once)
  - using function (function call . many)

```c
// create function
int myfunction(int a, int b){
  return a+b;
}

int main(){
  int c;
  // function call
  c = myfunction( 2, 3);
  printf("%d\n", c)
}
```

```c
// predefine
int myfunction(int a, int b);

int main(){
  int c;
  // function call
  c = myfunction( 2, 3);
  printf("%d\n", c)
}

// create function
int myfunction(int a, int b){
  return a+b;
}
```

Functions

https://github.com/kkh3363/C_Lang

Parameters and Arguments

- Information can be passed to functions as a parameter. Parameters act as variables inside the function.
- Parameters are specified after the function name, inside the parentheses. You can add as many parameters as you want, just separate them with a comma:
- Syntax

```
returnType functionName(parameter1, parameter2, parameter3) {
  // code to be executed
}
```

- **void** means that the function does not have a return value.

Functions

https://github.com/kkh3363/C_Lang

Parameters and Arguments

- In the example below, the function takes a string of characters with name as parameter. When the function is called, we pass along a name, which is used inside the function to print "Hello" and the name of each person:

```c
void myFunction(char name[]) {
  printf("Hello %s\n", name);
}

int main() {
  myFunction("Liam");
  myFunction("Jenny");
  myFunction("Anja");
  return 0;
}

// Hello Liam
// Hello Jenny
// Hello Anja
```

Functions

https://github.com/kkh3363/C_Lang

Multiple Parameters

- Inside the function, you can add as many parameters as you want:

```c
void myFunction(char name[], int age) {
  printf("Hello %s. You are %d years old.\n", name, age);
}

int main() {
  myFunction("Liam", 3);
  myFunction("Jenny", 14);
  myFunction("Anja", 30);
  return 0;
}

// Hello Liam. You are 3 years old.
// Hello Jenny. You are 14 years old.
// Hello Anja. You are 30 years old.
```

Functions

https://github.com/kkh3363/C_Lang

Pass Arrays as Function Parameters

- You can also pass arrays to a function:

```
void myFunction(int myNumbers[5]) {
  for (int i = 0; i < 5; i++) {
    printf("%d\n", myNumbers[i]);
  }
}

int main() {
  int myNumbers[5] = {10, 20, 30, 40, 50};
  myFunction(myNumbers);
  return 0;
}
```

Functions

https://github.com/kkh3363/C_Lang

Return Values

- The void keyword, used in the previous examples, indicates that the function should not return a value.
- If you want the function to return a value, you can use a data type (such as int or float, etc.) instead of void, and use the return keyword inside the function:

```
int myFunction(int x) {
  return 5 + x;
}

int main() {
  printf("Result is: %d", myFunction(3));
  return 0;
}

// Outputs 8 (5 + 3)
```

Functions

https://github.com/kkh3363/C_Lang

- Example
  - To demonstrate a practical example of using functions, let's create a program that converts a value from fahrenheit to celsius:

```c
// Function to convert Fahrenheit to Celsius
float toCelsius(float fahrenheit) {
  return (5.0 / 9.0) * (fahrenheit - 32.0);
}

int main() {
  // Set a fahrenheit value
  float f_value = 98.8;

  // Call the function with the fahrenheit value
  float result = toCelsius(f_value);

  // Print the fahrenheit value
  printf("Fahrenheit: %.2f\n", f_value);

  // Print the result
  printf("Convert Fahrenheit to Celsius: %.2f\n", result);

  return 0;
}
```

Function

https://github.com/kkh3363/C_Lang

- Challenge 1
  - Exchange rate calculate
  - Input dollars from key board , output the equivalent in Korea Won
  - 1$ -> 1,500 won
  - Ex)

    Input dollars : 2

    3000 won
- Challenge 2
  - A program that calculates how many dollars are needed to buy coffee
  - 1 cup of coffee is 3,000 won

2. Variable Scope

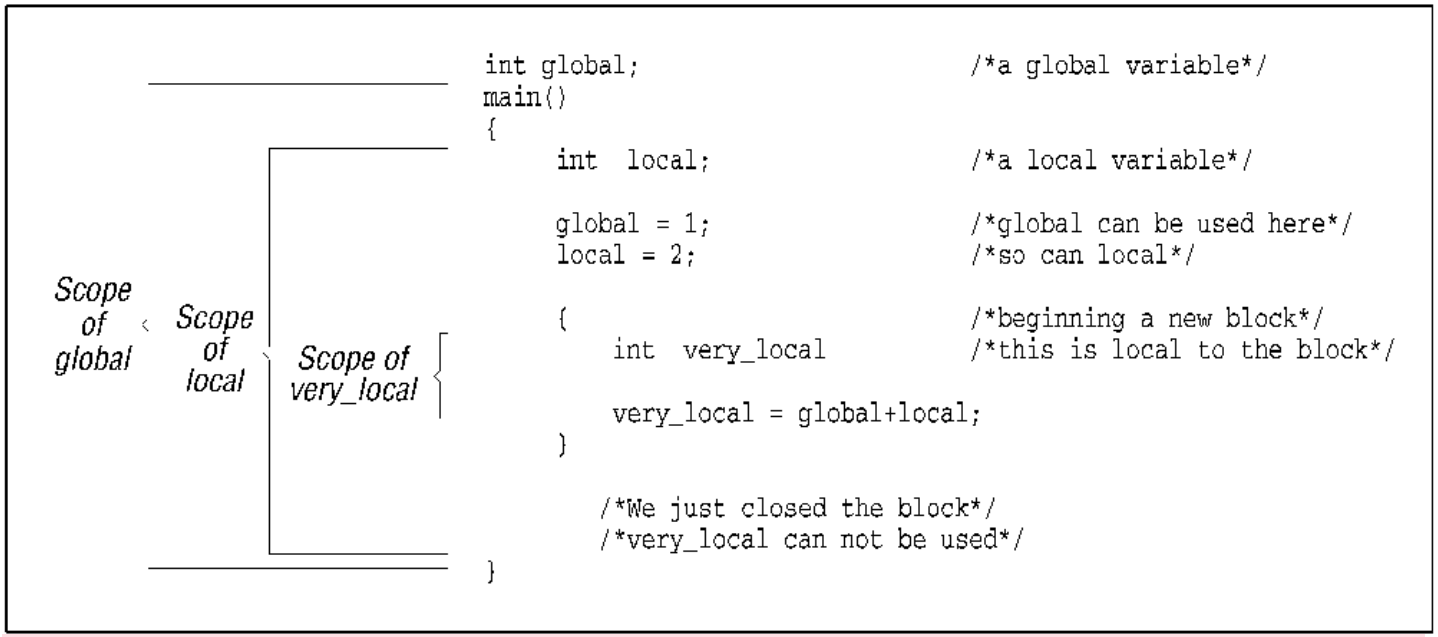
Variable Scope

https://github.com/kkh3363/C_Lang

- Scope
  - It is important to learn how variables act inside and outside of functions.
  - In C, variables are only accessible inside the region they are created.
    This is called ***scope***.

Variable Scope

https://github.com/kkh3363/C_Lang



Local Variable	Global Variable
Local variables are declared inside a function.	Global Variables are declared before the main function.
Local Variables cannot be accessed outside the function.	Global Variables can be accessed in any function.
Local Variables are alive only for a function.	Global Variables are alive till the end of the program.

Variable Scope

https://github.com/kkh3363/C_Lang

- Example

```
#include <stdio.h>

// Global variable
int x = 5;

void myFunction() {
  printf("%d\n", ++x);  // Increment the value of x by 1 and print it
}

int main() {
  myFunction();

  printf("%d\n", x); // Print the global variable x
  return 0;
}
```

Variable Scope

https://github.com/kkh3363/C_Lang

- Naming Variables
  - If you operate with the same variable name inside and outside of a function, C will treat them as two separate variables; One available in the global scope (outside the function) and one available in the local scope (inside the function):

```c
// Global variable x
int x = 5;

void myFunction() {
  // Local variable with the same name as the global variable (x)
  int x = 22;
  printf("%d\n", x); // Refers to the local variable x
}

int main() {
  myFunction();

  printf("%d\n", x); // Refers to the global variable x
  return 0;
}
```

Variable Scope

https://github.com/kkh3363/C_Lang

- Naming Variables
  - However, you should avoid using the same variable name for both globally and locally variables as it can lead to errors and confusion.
  - In general, you should be careful with global variables, since they can be accessed and modified from any function:

```c
// Global variable
int x = 5;

void myFunction() {
  printf("%d\n", ++x); // Increment the value of x by 1 and print it
}

int main() {
  myFunction();

  printf("%d\n", x); // Print the global variable x
  return 0;
}

// The value of x is now 6 (no longer 5)
```

Function

https://github.com/kkh3363/C_Lang

- Function Declaration and Definition
  - A function consist of two parts:
    - Declaration: the function's name, return type, and parameters (if any)
    - Definition: the body of the function (code to be executed)

```
void myFunction() { // declaration
  // the body of the function (definition)
}
```

Function

https://github.com/kkh3363/C_Lang

- Function Declaration and Definition
  - For code optimization, it is recommended to separate the declaration and the definition of the function.
  - You will often see C programs that have function declaration above main(), and function definition below main().
  - This will make the code better organized and easier to read

```c
// Function declaration
void myFunction();

// The main method
int main() {
  myFunction();  // call the function
  return 0;
}

// Function definition
void myFunction() {
  printf("I just got executed!");
}
```

Function

https://github.com/kkh3363/C_Lang

- Parameters
  - If we use the example from the function parameters chapter regarding parameters and return values
  - It is considered good practice to write it like this instead

```c
int myFunction(int x, int y) {
  return x + y;
}

int main() {
  int result = myFunction(5, 3);
  printf("Result is = %d", result);
  return 0;
}
// Outputs 8 (5 + 3)
```

```c
// Function declaration
int myFunction(int x, int y);

// The main method
int main() {
  int result = myFunction(5, 3); // call the function
  printf("Result is = %d", result);
  return 0;
}

// Function definition
int myFunction(int x, int y) {
  return x + y;
}
```

Function

https://github.com/kkh3363/C_Lang

- Separate user function
  - myFunction.h
  - myFunction.c
  - main.c

```c
// myfunc.h
int myFunction(int x, int y);
```

```c
// myfunc.c
#include "myFunction.h"

int myFunction(int x, int y){
  return x + y;
}
```

```c
// main.c
#include "myFunction.h"

int main() {
  int result = myFunction(5, 3);
  printf("Result is = %d", result);
  return 0;
}
```

Function

https://github.com/kkh3363/C_Lang

- Challenge
  - Create a calculator program
    1. input operator from standard input(keyboard)
    2. input numbers from standard input
    3. return result.

```
#include <stdio.h>

int main() {
		double num1, num2;
		char operator;
		double result;

		printf("insert exp :");
		scanf("%lf %c %lf", &num1, &operator, &num2);
		switch (operator) {
				case '+': result = num1 + num2;break;
				case '-': result = num1 - num2; break;
				case '*': result = num1 * num2; break;
				case '/': result = num1 / num2; break;
		}
		printf("%.1f %c %.1f = %.1f ₩n", num1, operator, num2, result);
		return 0;
}
```

```
#include <stdio.h>
double add(double a, double b) { return a + b; }
double sub(double a, double b) {return a - b;}
double mul(double a, double b) { return a * b; }
double div(double a, double b) {  return a / b;}
int main() {
		double num1, num2;
		char operator;
		double result;
		printf("insert exp :");
		scanf("%lf %c %lf", &num1, &operator, &num2);
		switch (operator) {
				case '+': result = add(num1, num2);break;
				case '-': result = sub(num1, num2); break;
				case '*': result = mul(num1, num2); break;
				case '/': result = div(num1, num2); break;
		}
		printf("%.1f %c %.1f = %.1f ₩n", num1, operator, num2,
result);
		return 0;
}
```

Math Function

https://github.com/kkh3363/C_Lang

Math Function

- There is also a list of math functions available, that allows you to perform mathematical tasks on numbers.
- https://www.w3schools.com/c/c_ref_math.php
- To use them, you must include the math.h header file in your program:

```
#include <math.h>
// Square Root
printf("%f", sqrt(16));
```

- Round a Number
  - The ceil() function rounds a number upwards to its nearest integer, and the floor() method rounds a number downwards to its nearest integer, and returns the result:

```
printf("%f", ceil(1.4));
printf("%f", floor(1.4));
```

Inline Function

https://github.com/kkh3363/C_Lang

Inline Function

- You might sometimes see the inline keyword used in other people's functions. It's not something you need to use often as a beginner, but it's good to know what it means.
- An inline function is a small function that asks the compiler to insert its code directly where it is called, instead of jumping to it.
- This can make short, frequently used functions a little faster, because it removes the small delay of a normal function call.
- Let's compare a regular function with an inline function:

Regular Function

```
int square(int x) {
  return x * x;
}
```

Inline Function

```
inline int square(int x) {
  return x * x;
}
```

- Both functions work the same way. The only difference is that the inline version suggests to the compiler to copy the function's code directly where it is used.

Inline Function

https://github.com/kkh3363/C_Lang

When Not to Use Inline

- Inline functions are best for small, simple functions. Avoid using them for:
  - Large functions (they make your program bigger)
  - Recursive functions
  - Functions that are rarely called
- Too many inline functions can make your program slower and larger, a problem known as code bloat

Regular Function	Inline Function
Code jumps to the function each time it's called	Code is inserted directly where it's called
Slightly slower (small delay)	Slightly faster
Good for large functions	Good for small functions

Recursion

https://github.com/kkh3363/C_Lang

- Recursion
  - Recursion is the technique of making a function call itself.
  - This technique provides a way to break complicated problems down into simple problems which are easier to solve.
  - Recursion may be a bit difficult to understand. The best way to figure out how it works is to experiment with it.

Recursion

https://github.com/kkh3363/C_Lang

- Recursion Example
  - Adding two numbers together is easy to do, but adding a range of numbers is more complicated. In the following example, recursion is used to add a range of numbers together by breaking it down into the simple task of adding two numbers:

```
10 + sum(9)
10 + ( 9 + sum(8) )
10 + ( 9 + ( 8 + sum(7) ) )
...
10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 + 1 + sum(0)
10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 + 1 + 0
```

```c
int sum(int k);

int main() {
  int result = sum(10);
  printf("%d", result);
  return 0;
}

int sum(int k) {
  if (k > 0) {
    return k + sum(k - 1);
  } else {
    return 0;
  }
}
```

Recursion

https://github.com/kkh3363/C_Lang

- Recursion Example
  - Use recursion to count down from 5:

```c
void countdown(int n);

int main() {
  countdown(5);
  return 0;
}

void countdown(int n) {
  if (n > 0) {
    printf("%d ", n);
    countdown(n - 1);
  }
}
```

Recursion

https://github.com/kkh3363/C_Lang

- Calculate Factorial with Recursion
  - This example uses a recursive function to calculate the factorial of 5

```c
int factorial(int n);

int main() {
  printf("Factorial of 5 is %d", factorial(5));
  return 0;
}

int factorial(int n) {
  if (n > 1) {
    return n * factorial(n - 1);
  } else {
    return 1;
  }
}
```

Function Pointer

https://github.com/kkh3363/C_Lang

Function Pointer

- A function pointer is like a normal pointer, but instead of pointing to a variable, it points to a function.
- This means it stores the address of a function, allowing you to call that function using the pointer.
- Function pointers let you decide which function to run while the program is running, or when you want to pass a function as an argument to another function.
- Think of it like saving a phone number - the pointer knows where the function lives in memory, so you can "call" it later.

Function Pointer

https://github.com/kkh3363/C_Lang

- Declaring a Function Pointer
  - The general syntax for declaring a function pointer is :

```
returnType (*pointerName)(parameterType1, parameterType2, ...);
```

- Assigning a Function to a Pointer
  - You can assign a function to a pointer in two ways:
  - Both are the same, because the function's name already represents its address in memory.

```
ptr = add;
ptr = &add;
```

- Calling a Function Through a Pointer
  - Once the pointer is assigned, you can call the function in two ways:

```
ptr(5, 3);
(*ptr)(5, 3);
```

https://github.com/kkh3363/C_Lang

```c
#include <stdio.h>

int add(int a, int b) {
    return a + b;
}
int sub(int a, int b) {
    return a - b;
}

int main() {
    int x, y;
    int result;
    char op;
    printf("insert operation (+ or -) :");
    scanf("%c", &op);
    printf("insert num1 num2 :");
    scanf("%d %d", &x, &y);
    switch (op) {
    case '+': result = add(x, y); break;
    case '-': result = sub(x, y); break;
    }
    printf(" % d % c % d = % d\n", x, op, y, result);
    return 0;
}
```

```c
#include <stdio.h>
int add(int a, int b) {
    return a + b;
}
int sub(int a, int b) {
    return a - b;
}
int main() {
    int x, y;
    int result;
    char op;
    int (*operator)(int a, int b);

    printf("insert operation (+ or -) :");
    scanf("%c", &op);
    printf("insert num1 num2 :");
    scanf("%d %d", &x, &y);
    switch (op) {
    case '+': operator = add ; break;
    case '-': operator = sub; break;
    }
    result = operator(x, y);
    printf(" % d % c % d = % d\n", x, op, y, result);
    return 0;
}
```

Function Pointer

https://github.com/kkh3363/C_Lang

Function Pointer Example

- Now that you know how to declare and assign a function pointer, let's see a complete example:
- ptr is a pointer to the function add().
- ptr(5, 3) calls the function using the pointer.
- This works exactly the same as calling add(5, 3).

```c
int add(int a, int b) {
  return a + b;
}

int main() {
  int (*ptr)(int, int) = add;
  int result = ptr(5, 3);
  printf("Result: %d", result);
  return 0;
}
```

Function Pointer

https://github.com/kkh3363/C_Lang

- Passing a Function as an Argument
  - Function pointers can be passed to other functions - this is called a ***callback***.
  - It allows a function to call another function that you provide as input.

```c
void greetMorning() { printf("Good morning!\n"); }
void greetEvening() { printf("Good evening!\n"); }

void greet(void (*func)()) {
  func();
}

int main() {
  greet(greetMorning);
  greet(greetEvening);
  return 0;
}
```

Function Pointer

https://github.com/kkh3363/C_Lang

- Function Pointer Array
  - You can also store several function pointers in an array, so you can choose which function to run while the program is running.
  - This is often used for simple menus, command lists, or calculators - anywhere you want to call different functions based on user input.

```c
void add(int a, int b) { printf("Result: %d\n", a + b); }
void subtract(int a, int b) { printf("Result: %d\n", a - b); }
void multiply(int a, int b) { printf("Result: %d\n", a * b); }

int main() {
  int choice, x = 10, y = 5;

  void (*operations[3])(int, int) = { add, subtract, multiply };

  printf("x = %d, y = %d\n\n", x, y);
  printf("Choose an operation:\n");
  printf("0: Add\n1: Subtract\n2: Multiply\n");
  scanf("%d", &choice);

  if (choice >= 0 && choice < 3) {
    operations[choice](x, y);
  } else {
    printf("Invalid choice!\n");
  }

  return 0;
}
```

Function Pointer

https://github.com/kkh3363/C_Lang

Normal Function	Function Pointer
Called directly by its name	Called using a pointer
The function is decided before the program runs	You can choose which function to call while the program is running
Good for simple code	Good for flexible and reusable code

Function Pointer

https://github.com/kkh3363/C_Lang

- Callback Function
  - A ***callback function*** is a function that is passed as an argument to another function.
  - The receiving function can then ***call it back*** (run it) whenever it needs to.
  - This is a powerful way to make your code ***flexible and reusable*** - you can decide which function should run, without changing the main logic.
  - In C, callback functions are usually implemented using ***function pointers***.

Function Pointer

https://github.com/kkh3363/C_Lang

- Simple Callback Example

```c
void sayHello() {
  printf("Hello from the callback!\n");
}

void runCallback(void (*callback)()) {
  printf("Before calling the callback...\n");
  callback();
  printf("After calling the callback.\n");
}

int main() {
  runCallback(sayHello);
  return 0;
}
```

Function Pointer

https://github.com/kkh3363/C_Lang

- Callback with Parameters
  - You can also pass functions that take parameters - just make sure the function pointer type matches

```c
void addNumbers(int a, int b) {
  printf("The sum is: %d\n", a + b);
}

void calculate(void (*callback)(int, int), int x, int y) {
  callback(x, y);
}

int main() {
  calculate(addNumbers, 5, 3);
  return 0;
}
```

Function Pointer

https://github.com/kkh3363/C_Lang

- Using Callbacks in qsort()
  - Many C standard library functions use callbacks. For example, the qsort() function in <stdlib.h> uses a callback to compare elements while sorting.
  - You provide the comparison function, and qsort() calls it as needed. This will sort the elements:

```c
#include <stdio.h>
#include <stdlib.h>

int compare(const void *a, const void *b) {
  return (*(int*)a - *(int*)b);
}

int main() {
  int numbers[] = { 5, 2, 9, 1, 7 };
  int size = sizeof(numbers) / sizeof(numbers[0]);

  qsort(numbers, size, sizeof(int), compare);

  for (int i = 0; i < size; i++) {
    printf("%d ", numbers[i]);
  }
  return 0;
}
```

BAEKSEOK UNIVERSITY

Thank You !